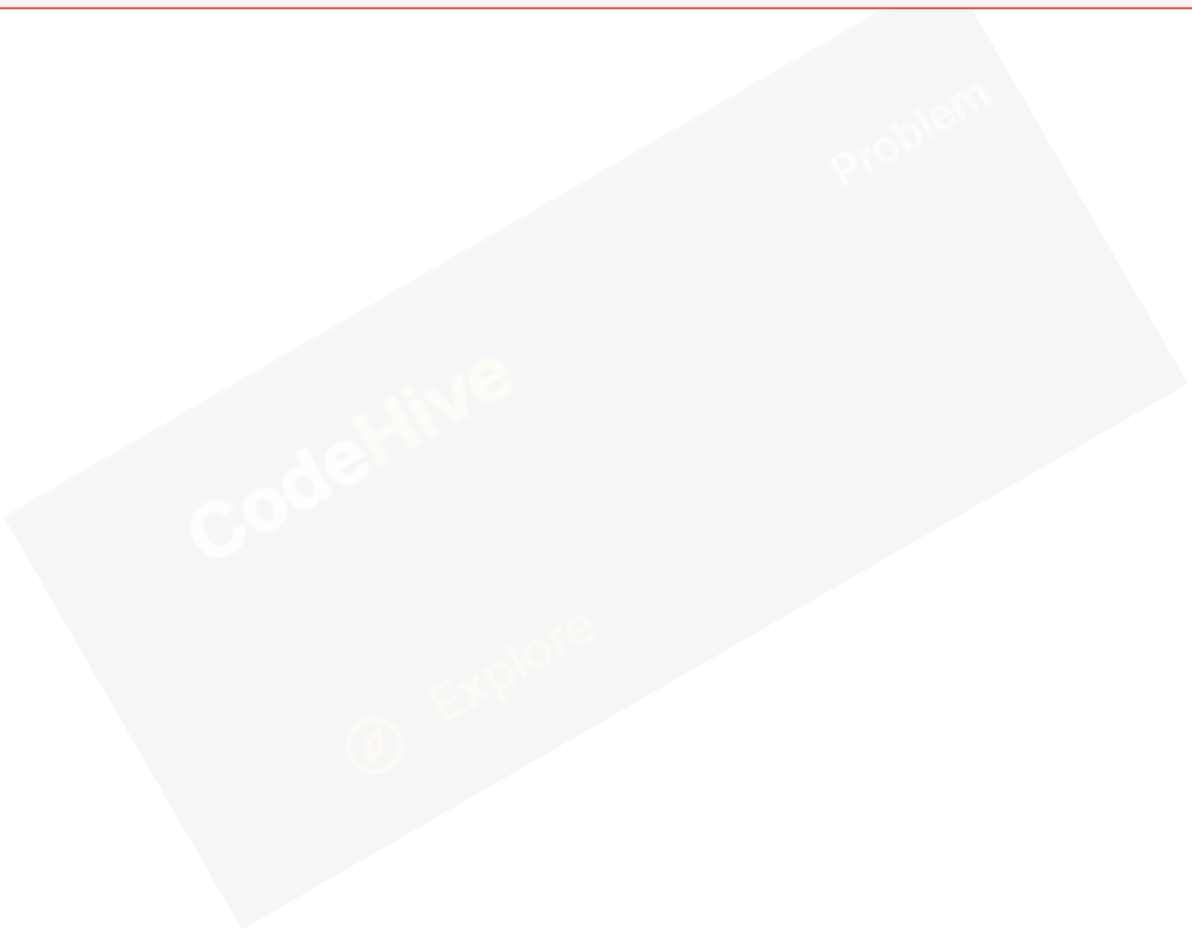


Memory Management in C

Memory management in C refers to the process of allocating, using, and freeing memory efficiently during program execution.

One-Line Definition: *Memory management in C is the process of allocating and freeing memory dynamically during program execution to optimise resource usage.*



1. The Memory Segments

A C program's memory is divided into specific areas:

- **Stack:** Automated, temporary storage.
- **Heap:** Manual, dynamic storage.
- **Static/Global:** Lifetime of the program.
- **Code Segment:** Stores compiled instructions.



2. Stack Memory

The stack is managed automatically by the compiler. When a function is called, a "stack frame" is created; when it returns, the frame is destroyed.

- **Speed:** Very fast.
- **Limit:** Small and fixed size (can cause Stack Overflow).
- **Variables:** Local variables and function parameters.



3. Heap Memory

The Heap is used for **Dynamic Memory Allocation**. It allows you to request memory when the program is running, especially when you don't know the size of data beforehand.

Unlike the stack, the Heap has no automatic cleanup. You must manage it yourself.



4. malloc()

malloc stands for *Memory Allocation*. It reserves a block of raw memory on the heap.

```
int *ptr = (int*)malloc(5 * sizeof(int));
```

- Returns a void* pointer.
- Values are **uninitialized** (garbage values).

5. calloc()

calloc stands for *Contiguous Allocation*. It is similar to malloc but with two key differences:

```
int *ptr = (int*)calloc(5, sizeof(int));
```

- Takes two arguments (number of elements, size of each).
- **Initializes all bytes to zero.**

6. realloc()

If the previously allocated memory is insufficient or too much, `realloc` allows you to resize the block.

```
ptr = realloc(ptr, 10 * sizeof(int));
```

It attempts to expand the current block or moves the data to a new location if necessary.

CodeHive

@ Explore

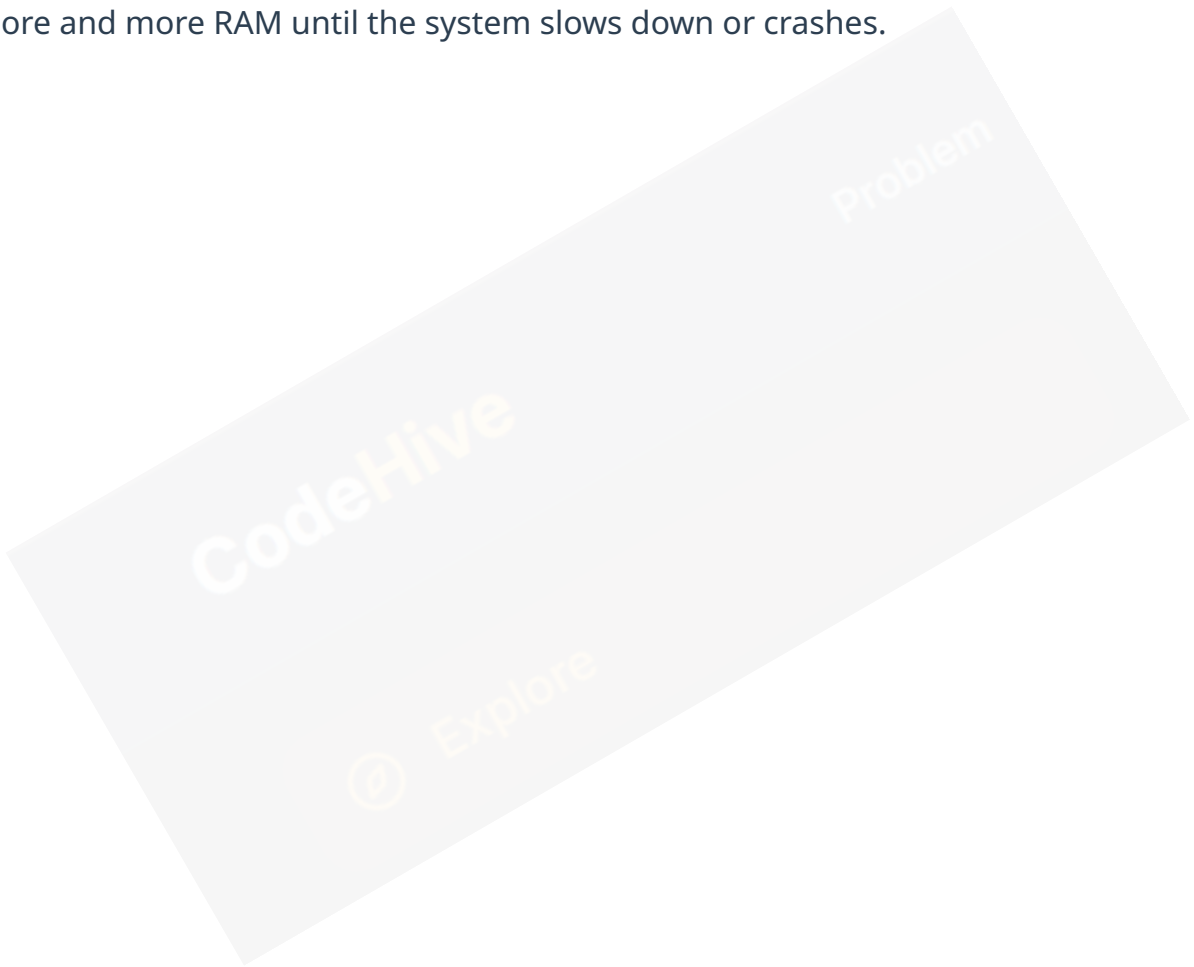
Problem

7. free()

This is the most important function. It releases the memory back to the system.

```
free(ptr);
```

Failing to use `free()` leads to **Memory Leaks**, where your program consumes more and more RAM until the system slows down or crashes.



8. Security: The NULL Check

Dynamic allocation can fail if the system is out of memory. Professional code always checks the pointer:

```
int *p = malloc(100);
if (p == NULL) {
    printf("Memory Allocation Failed!");
    exit(1);
}
```

CodeHive

@ Explore

9. Function Comparison

Function	Initial Value	Common Use
malloc	Garbage	General allocation
calloc	Zero	Arrays / Clean start
realloc	Preserved	Changing buffer size

10. Common Pitfalls

- **Dangling Pointers:** Accessing memory after calling `free()`.
- **Memory Leaks:** Losing the pointer to a heap block without freeing it.
- **Buffer Overflow:** Writing past the end of the allocated block.



11. CodeHive Rules

1. Always set your pointer to NULL after `free(ptr)`.
2. Always use `sizeof`; never hardcode byte sizes like 4 or 8.
3. Match every `malloc` with exactly one `free`.



Key Points (Exam Ready)

- Header file required: `<stdlib.h>`.
 - Heap memory is persistent until manually freed.
 - **malloc** is faster but **calloc** is safer (avoids garbage).
 - Dynamic memory is the foundation of Linked Lists and Trees.
-

Master Memory - CodeHive Official

CodeHive

@ Explore